



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

配置文件与路由

WebServer V1.4: 配置驱动

从 V1.3 到 V1.4

V1.3 已能处理静态资源、GET 和 POST，但关键行为仍写死在代码中。

- 修改监听地址或端口，需要改代码并重新编译。
- 新增路由，需要继续增加 if/else。
- 日志路径和等级无法按环境调整。

V1.4：启动时读取配置，校验后构建服务器参数和路由表。



本次实训目标

完成 WebServer V1.4 后，应能够：

- 使用成熟 JSON 库读取 server.json。
- 校验 host、port、document_root、日志和 routes。
- 根据 method + path 构建路由表并调用 handler。
- 正确返回 404、405，并保留 V1.3 的 GET、POST 与静态资源功能。



配置描述数据，不执行代码

配置文件只保存需要按环境调整的数据：

- 监听地址、端口和网站根目录。
- 日志等级和日志文件。
- 路由的 method、path 与 handler 名称。

配置文件不保存函数地址或任意可执行代码。handler 名称必须在程序内注册表中查找，未知名称属于配置错误。



为什么本项目统一使用 JSON

格式没有绝对优劣。本实训统一使用 JSON，减少环境差异，把精力放在加载、校验和路由分发。

INI	简单，但嵌套结构弱
Properties	键值直观，更偏 Java 生态
XML	表达能力强，但内容冗长
YAML	易读，但缩进敏感
JSON	结构清晰、解析库成熟 <- 本项目采用



server.json 描述完整启动参数

配置文件描述服务器参数和路由元数据，不包含函数地址。下面字段贯穿后续实现与验收。

```
{
  "server": {
    "host": "127.0.0.1", "port": 8080,
    "document_root": "./www"
  },
  "logging": {"level": "INFO",
    "file": "./logs/server.log"},
  "routes": [
    {"method": "GET", "path": "/users", "handler": "users_get"},
    {"method": "POST", "path": "/users", "handler": "users_create"}
  ]
}
```



先校验配置，再启动监听

所有配置必须在 bind()、listen() 之前完成加载与校验，避免服务器带着错误状态运行。

```
./mini_webserver ./config/server.json
```

读取文件 -> JSON 解析 -> 字段校验

Config 对象 -> handler 映射 -> 路由表

create_listener(host, port)

进入 epoll / kqueue 事件循环



配置校验要在启动阶段失败

必须校验：

- host 可绑定，port 范围为 1 ~ 65535。
- document_root 存在、可读且是目录。
- logging.level 只能取 DEBUG、INFO、WARN、ERROR。
- method 合法，path 以 / 开头，(method, path) 不重复。
- handler 名称已经在程序注册表中注册。

关键字段错误时：打印字段名和原因，服务器不进入监听状态。



路由键是 method + path

同一路径可以由不同 HTTP 方法执行不同逻辑；查询字符串不参与路由匹配。

请求	路由键
GET /users?id=7	GET /users
POST /users	POST /users
GET /users	-> users_get
POST /users	-> users_create
path 不存在	-> 404
path 存在但 method 不允许	-> 405 + Allow



handler 名称通过注册表绑定函数

配置中的 handler 只是名称；程序通过注册表查找函数，未知名称则启动失败。

```
typedef int (*Handler)(const HttpRequest *, HttpResponse *);
typedef struct { const char *name; Handler fn; } HandlerEntry;
static const HandlerEntry registry[] = {
    {"users_get", users_get},
    {"users_create", users_create}
};
Handler find_handler(const char *name) {
    size_t n = sizeof registry / sizeof registry[0];
    for (size_t i = 0; i < n; ++i) {
        if (strcmp(name, registry[i].name) == 0)
            return registry[i].fn;
    }
    return NULL;
}
```



请求分发只做一次明确决策

收到完整请求后，先规范化 path，再根据 method + path 查找路由并生成响应。

```
handle_request(req):  
    path = normalize_path(req.target)  
    route = find_route(req.method, path)  
  
    if route exists:  
        return route.handler(req)  
    if method_not_allowed(path):  
        return 405 + Allow header  
    if req.method == GET:  
        return try_static_file(path)  
    return 404 Not Found
```



V1.4 不能破坏 V1.3 的行为

V1.4 只改变“参数和路由从哪里来”，已有 HTTP 行为必须保持一致。

- 配置成功：输出 host、port 和路由数量后进入事件循环。
- 目标存在且方法允许：返回 200 或相应业务结果。
- 没有动态路由且没有静态资源：返回 404。
- path 存在但 method 不允许：返回 405，并携带 Allow。
- 配置缺失、JSON 错误或字段非法：启动失败，不进入事件循环。



实训内容

在 WebServer V1.3 基础上，完成 WebServer V1.4。

1. 新建 config/server.json，服务器启动时读取并解析。
2. 配置文件至少包含：
 - 服务器绑定的 IP 和端口
 - www 静态资源根目录
 - 日志等级和日志路径
 - GET、POST 路由
3. 修改配置并重启后，新参数应生效。
4. 配置错误时停止启动并输出原因。
5. WebServer V1.3 原有功能继续可用。



最低验收证据

1. 修改配置中的 port，重启后监听新端口；源代码无需改动。
2. GET /users 与 POST /users 命中不同 handler。
3. 未知 path 返回 404；错误 method 返回 405 并包含 Allow。
4. 配置缺失、JSON 错误、重复路由或未知 handler 时拒绝启动，并打印原因。
5. 首页、静态资源及 V1.3 的 GET/POST 测试仍然通过。

总结：配置负责描述，注册表负责绑定，路由表负责分发。